Elenco esercizi C proposti nelle prove scritte

Enrico Bini Massimiliano De Pierro Rossano Gaeta

29 ottobre 2019

Indice

2019.09.13, corso A **2**

2019.09.13, corso B **2**

2019.07.15, corso A **3**

2019.07.15, corso B **3**

2019.06.18, corso A **4**

2019.06.18, corso B **5**

2019.02.18, corso A **5**

2019.02.18, corso B **6**

2019.01.28, corso A **6**

2019.01.28, corso B **7**

2018.09.06, corso A **8**

2018.09.06, corso B **8**

2018.07.09, corso B **9**

2018.06.28, corso A **10**

2018.06.28, corso B **11**

2018.06.15, corso A **12**

2018.02.13, corso A **13**

2018.02.13, corso B **13**

2018.01.22, corso A **14**

2018.01.22, corso B **15**

2019.09.13, corso A

Esercizio 1 (5 punti)

Si implementi la funzione con prototipo

```
char * ins_substring(char * s, char sel, char * ins);
```

i cui parametri sono:

- una stringa `s` (terminata dal byte 0 come di consueto),
- un carattere `sel`,
- una stringa `ins` da insesire.

La funzione alloca e ritorna una nuova stringa costruita a partire da `s`, sostituendo ad ogni occorrenza del carattere `sel` la stringa `ins`. Se `sel` non è presente in `s`, viene restituita una copia di `s`. Se, ad esempio, viene invocato `ins_substring("Ciao a tutti",'i',"123")` viene ritornata la stringa `"C123ao a tutt123"`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
              int value;
              struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link cut(link head, int begin, int end);
```

dove: `head` è un puntatore alla testa di una lista; `begin` e `end` rappresentano due posizioni all'interno della lista a partire da `head`, con `begin<=end`. Le posizioni sono numerare a partire da 0. La funzione deve restituire, se possibile, la sotto-lista compresa tra le posizioni `begin` e `end` deallocando la rimanente parte. La funzione restituisce il puntatore alla testa della lista così calcolata o `NULL` se non è possibile. Ad esempio se `head->{4}->{7}->{3}->{8}->{2}->{1}` allora `cut(head,2,4)` restituisce la lista `{3}->{8}->{2}`. Se `end` va oltre il termine della lista la funzione deve restituire la sottolista tra `begin` e la fine della lista.

2019.09.13, corso B

Esercizio 1 (5 punti)

Si implementi la funzione con prototipo

```
char * get_sel(char * s, char sel);
```

i cui parametri sono:

- una stringa `s` (terminata dal byte 0 come di consueto) e
- un carattere `sel`.

La funzione alloca e ritorna la sotto-stringa di `s` compresa fra la prima occorrenza di `sel` e la seconda. Se è presente una sola occorrenza di `sel` ritorna la sottostringa fino alla fine di `s`. Se `s` non contiene alcun carattere `sel`, ritorna `NULL`. Se, ad esempio, viene invocato `get_sel("Italia Italia",'i')` viene ritornata la stringa `"a Ital"`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
              int value;
              struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link insert(link head, link patch, int pos);
```

la quale inserisca nella lista puntata da `head` la lista puntata da `patch`, a partire dala posizione `pos` (le posizioni sono numerate a partire da 0). La funzione restituisce il puntatore alla lista così modificata. Ad esempio, se `head->{4}->{7}->{3}->{8}` e `patch->{9}->{2}` allora `insert(haed, patch,2)` restituisce la lista `head->{4}->{7}->{3}->{9}->{2}->{8}`. In caso di valori inconsistenti degli argomenti la funzione deve restituire `NULL` (ad esempio se `pos` va oltre il termine della lista `head` o è negativo).

2019.07.15, corso A

Esercizio 1 (5 punti)

Si consideri la seguente struttura C di nome `sample`:

```
struct sample { int     value;
                double  time; };
```

essa descrive genericamente una *campionatura* (`struct sample`) composta da un campo valore (`value`) e da un campo tempo (`time`).

Implementare la funzione con prototipo

```
int get_value(struct sample recordings[], int size, double nearest);
```

La funzione `get_value` ha tre parametri:

- `recordings` è un array non ordinato di *campionature*,
- `size` specifica la dimensione dell'array,
- `nearest` rappresenta un istante di tempo in secondi.

La funzione deve restituire il campo `value` della campionatura registrata in `recordings` prossima al tempo `nearest` (più vicina). In caso tale valore non possa essere determinato, la funzione imposta la variabile globale di nome `errno` di tipo `int` e definita in un file esterno al valore 256.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link roll(link head);
```

la quale modifica e ritorna la lista puntata da `head` in cui l'elemento di testa (puntato da `head`) viene spostato in ultima posizione (in coda). Se per esempio, viene invocata `roll(...)` con la lista $3 \to 8 \to 2 \to 1$, viene ritornata la lista $8 \to 2 \to 1 \to 3$.

2019.07.15, corso B

Esercizio 1 (5 punti)

Si consideri la seguente struttura C di nome `sample`:

```
struct sample { int     value;
                double  time; };
```

essa descrive genericamente una *campionatura* (`struct sample`) composta da un campo valore (`value`) e da un campo tempo (`time`).

Implementare la funzione con prototipo

```
double get_mean(struct sample recordings[], int size);
```

La funzione `get_mean` ha due parametri:

- `recordings` è un array non ordinato di *campionature*,
- `size` specifica la dimensione dell'array,

La funzione deve restituire il valore medio delle campionature registrate in `recordings`. In caso tale valore non possa essere determinato, la funzione imposta la variabile globale di nome `errno` di tipo `int` e definita in un file esterno al valore 256.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link cut_min(link head);
```

la quale modifica e ritorna la lista puntata da `head` in cui viene tolto e deallocato l'elemento con campo `value` minimo. Se per esempio, viene invocata `cut_min(...)` con la lista $3 \to 8 \to 2 \to 1$, viene ritornata la lista $3 \to 8 \to 2$.

2019.06.18, corso A

Esercizio 1 (5 punti)

Si consideri la seguente struttura C di nome `record`:

```
struct record {
        int   *data;
        int   length;};
```

Essa descrive un generico array di dimensione `length` di `int` che permette l'accesso ai dati attraverso il puntatore `data`.

Implementare la funzione con prototipo

```
int * find_record(struct record recordings[], int n, int value);
```

Tale funzione ha tre parametri formali: `recordings` è un array di strutture `struct record` (**non** necessariamente una singola struttura `struct record`), `n` è la dimensione di `recordings` e `value` è un valore da cercare.

La funzione `find_record` deve restituire il puntatore `data` al primo array di `int` in `recordings` contenente tra i propri valori un elemento di valore `value`. Se non esiste alcun array contenente `value` la funzione deve resituire `NULL`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link head_elems(link head, int cut_value);
```

la quale elimini e deallochi tutti gli elementi della lista puntata da `head` a partire dalla prima occorrenza di un elemento con campo `value` uguale a `cut_value`. Il puntatore alla lista così modificata viene ritornato.

2019.06.18, corso B

Esercizio 1 (5 punti)

Si consideri la seguente struttura C di nome `record`:

```
struct record {
        int   *data;
        int   length;};
```

Essa descrive un generico array di dimensione `length` di `int` che permette l'accesso ai dati attraverso il puntatore `data`.

Implementare la funzione con prototipo

```
int erase_record_sum(struct record *recordings[], int n, int sum);
```

Tale funzione ha tre parametri formali: `recordings` è un array di strutture `struct record` (**non** necessariamente una singola struttura `struct record`), `n` è la dimensione di `recordings` e `sum` è un valore di somma sulla base del quale alcuni record in `recordings` e i rispettivi array di `int` puntati da `data` possono essere eliminati.

La funzione elimina e dealloca correttamente tutti i record la cui somma eccede il valore `sum` passato come parametro. La funzione, inoltre, ritorna il numero di record eliminati.

Un record di `recordings` eliminato avrà campo `data` uguale a `NULL` e campo `length` uguale a `0`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link pick_odds(link head);
```

la quale alloca e ritorna una nuova lista i cui elementi hanno campo `value` uguale ai campi `value` degli elementi in posizione dispari della lista puntata da `head` (si consideri il primo elemento in posizione 1). Se per esempio, viene invocata `pick_odds(...)` con la lista $3 \to 8 \to 2 \to 1$, viene allocata e ritornata la lista $3 \to 2$.

2019.02.18, corso A

Esercizio 1 (5 punti)

Implementare la funzione con prototipo

```
int *greater_than(int * a, int * b, const int a_length, const int b_length);
```

la funzione ha quattro parametri formali rispettivamente di nomi `a`, `b`, `a_length` e `b_length`: `a` e `b` sono array di `int` e `a_length` e `b_length` le rispettive dimensioni.

La funzione deve restituire un nuovo array di `int`, di dimensioni `a_length` tale che il generico elemento di indice `i` conti quanti elementi di `b` sono maggiori di `a[i]`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link clean_odd(link head);
```

che modifica e ritorna la lista `head` da cui sono stati rimossi e deallocati i nodi che hanno campo `value` **dispari**.

Per esempio, se viene passata la lista i cui campi `value` sono:

`3 -> -1 -> 4 -> -2`

la lista ritornata è

`4 -> -2`

2019.02.18, corso B

Esercizio 1 (5 punti)

Implementare la funzione con prototipo

```
int *cumulate(int * a, const int a_length);
```

la funzione ha due parametri formali rispettivamente di nomi `a` e `a_length`: `a` è un array di `int` e `a_length` la rispettiva dimensione.

La funzione deve restituire un nuovo array di `int` tale che il generico elemento con indice `i` sia la somma cumulata di `a` alla posizione `i`, ossia se `r` è il nome dell'array da restituire allora: `r[i]` = `a[0]` + `a[1]` + ... `a[i]` per `i` = `0`...`a_length-1`.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link insert_in_middle(link head);
```

la quale prende in ingresso una **lista circolare** (ovvero il campo `next` dell'ultimo nodo è pari a `head`). Siano `min` e `max`, rispettivamente i valori minimo e massimo dei campi `value` della lista. La funzione `insert_in_middle(link head)` inserisce un nuovo nodo con campo `value` uguale a `(max+min)/2` dopo un nodo con campo `value` uguale a `min`. La funzione ritorna il puntatore al nodo inserito.

Per esempio, se viene passata la lista i cui campi `value` sono

`head -> 8 -> -1 -> 4 -> -2 -> (punta al nodo con valore 8)`

`max=8`, `min=-2` e quindi la lista viene modificata come segue

`head -> 3 -> 8 -> -1 -> 4 -> -2 -> (punta al nodo con valore 3)`

2019.01.28, corso A

Esercizio 1 (5 punti)

Implementare la funzione con prototipo

```
int range_of_even(int * nums, int length, int *a, int *b);
```

la funzione ha quattro parametri formali rispettivamente di nomi `nums`, `length`, `a` e `b`: `nums` è un array di numeri interi; `length` è la dimensione di `nums`; `a` e `b` sono puntatori usati dalla funzione per restituire degli interi al chiamante.

La funzione deve determinare il valore massimo e minimo **dei valori pari** presenti in `nums`. Se tali limiti esistono allora la funzione deve restituirli tramite i puntatori `a` e `b` al chiamante e restituire TRUE. Se l'array non contiene alcun numero pari, la funzione deve restituire FALSE e i valori in `*a` e `*b` non saranno significativi.

Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per tutti i parametri di tutte le funzioni richieste.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
link revert(link head);
```

che alloca e ritorna una nuova lista i cui elementi hanno campo `value` uguale a quelli della lista puntata da `head`, ma in **ordine inverso**.

Per esempio, se viene passata la lista i cui campi `value` sono:

```
3 -> -1 -> 4 -> -2
```

la lista ritornata è

```
-2 -> 4 -> -1 -> 3
```

2019.01.28, corso B

Esercizio 1 (5 punti)

Implementare la funzione con prototipo

```
int is_balanced(char * str, char **pos);
```

la funzione ha due parametri formali `str` e `pos`: `str` è un array di `char` nel quale è rappresentata una stringa C terminata dal byte `0`; `*pos` è utilizzato dalla funzione per restituire al chiamante un puntatore a `char`.

La funzione restituisce `TRUE` se e solo se nel testo rappresentato da `str` le parentesi tonde, eventualmente presenti, sono bilanciate: per ogni parentesi tonda che apre una sentenza deve esistere la controparte che chiude la sentenza. Le parentesi possono essere annidate le une nelle altre.

Inoltre se le parentesi non sono bilanciate la funzione restituisce tramite il parametro `pos` il puntatore al carattere di `str` al quale si è verificato l'errore, supponendo di leggere la stringa da sinistra a destra. Per esempio, se la stringa è "`ab(cde(fghi(jk)lmn`", `*pos` deve puntare all'ultimo carattere della stringa in quanto solo al raggiungimento del suo termine si può constatare che le parentesi non sono bilanciate; se la stringa è "`(ab(c))de)fgh`", `*pos` deve puntare alla parentesi tonda chiusa dopo il carattere `e` in quanto alla sua lettura si può determinare che le parentesi non sono bilanciate.

alla seconda parentesi aperta. Se le parentesi sono bilanciate il valore in `*pos` non è significativo.

Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per tutti i parametri di tutte le funzioni richieste.

Esercizio 2 (5 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
void accumulate(link head);
```

la quale modifica la lista puntata da `head` andando a sostituire al campo `value` di ogni elemento la somma dei campi `value` di tutti gli elementi dalla testa fino all'elemento corrente.

Per esempio, se viene passata la lista i cui campi `value`:

```
3 -> -1 -> 4 -> -2
```

la lista viene modificata come segue

```
3 -> 2 -> 6 -> 4
```

2018.09.06, corso A

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int verify_array(int * array1, int size1, int * array2, int size2);
```

che riceve in input i due vettori `array1` e `array2` di dimensioni rispettivamente `size1` e `size2`, e restituisce `TRUE` se e solo se ogni elemento del vettore `array2` è uguale alla somma di una qualsiasi coppia di elementi del vettore `array1`. La soluzione ***DEVE*** fare uso di una funzione di appoggio con prototipo

```
int check_pair(int element, int * array, int size);
```

che restituisce `TRUE` se e solo se esiste almeno una coppia di elementi del vettore `array` che sommati danno come risultato il parametro `element`. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per tutti i parametri di tutte le funzioni richieste.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdio.h>

void my_fun(char * s) {
	int a;

	a = s[0]-'0';
	a /= 2;
	if(a == (s[3]-'0')) {
		s[3] = *s;
	} else {
		s[3] = a+'0';
	}
	printf("%s\n", s+1);
}
```

- Se fosse invocata `my_fun(s)`, con il parametro `s` pari alla **stringa** contenente il tuo numero di matricola (correttamente terminata dal byte 0), cosa verrebbe stampato?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
	int value;
	struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int add_after(link head, int to_add, int after);
```

la quale aggiunga un nodo con campo `value` uguale al parametro `to_add` dopo il nodo in posizione `after` (si assuma che il primo elemento della lista si trovi in posizione 1). La funzione deve restituire `TRUE` se e solo se l'inserimento è avvenuto e `FALSE` altrimenti. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head` e per il parametro `after`.

2018.09.06, corso B

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int verify_vowel(char * string);
```

che restituisca `TRUE` se e solo se la stringa `string` è composta di sole vocali. La soluzione ***DEVE*** fare uso di una funzione di appoggio con prototipo

```
int is_vowel(char element);
```

che restituisca `TRUE` se e solo se il carattere `element` è una vocale. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per tutti i parametri di tutte le funzioni richieste.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdio.h>

int my_fun(char * s) {
        int a, b, c;

        /*
         * Si ricorda che la funzione sscanf e' analoga alla scanf
         * prelevando pero' l'input dalla stringa passata come primo
         * parametro piuttosto che da stdin
         */
        sscanf(s, "%d", &a);
        sscanf(s+1, "%d", &b);
        sscanf(s+2, "%d", &c);
        return a-b+c;
}
```

- Se fosse invocata `my_fun(s)`, con il parametro `s` pari alla **stringa** contenente il tuo numero di matricola (correttamente terminata dal byte 0), cosa verrebbe ritornato?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_after(link head, int after);
```

la quale cancelli il nodo che segue quello in posizione `after` (si assuma che il primo elemento della lista si trovi in posizione 1). La funzione deve restituire `TRUE` se e solo se la cancellazione è avvenuta e `FALSE` altrimenti. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head` e per il parametro `after`.

2018.07.09, corso B

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int controlla_punteggiatura(char * string);
```

che restituisca `TRUE` se e solo se nella stringa di caratteri `string` vi sono tanti punti esclamativi quanti sono i punti interrogativi. Si faccia uso di una funzione di appoggio che conti il numero di occorrenze di un carattere in una stringa. In ogni altro caso la funzione dovrà resituire `FALSE`. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per `string`.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <string.h>

signed char my_fun(signed char * v) {
	unsigned int len;

	len = strlen(v);
	if (len == 0)
		return 0;
	return v[0]-'7'+my_fun(v+1);
}
```

- Se fosse invocata `my_fun(s)`, con il parametro `s` pari alla **stringa** contenente il tuo numero di matricola, cosa verrebbe ritornato?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
	int value;
	struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_central_element(link head);
```

che lavori su una lista con almento tre elementi e la quale elimini l'elemento centrale se la lista contiene un numero dispari di elementi o i due elementi centrali se il numero di elementi della lista è pari. La funzione deve restituire `TRUE` nel primo caso e `FALSE` nel secondo caso. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head`.

2018.06.28, corso A

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int same_product(int * array, int size);
```

che restituisca `TRUE` se e solo se il prodotto tra il primo elemento dell'array e l'ultimo è uguale al prodotto tra il secondo ed il penultimo e così via. In ogni altro caso la funzione dovrà resituire `FALSE`. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per `array` e `size`. Specificare quale gestione si intende realizzare per il caso `size` dispari.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
char my_fun(unsigned int num) {
	signed char v[4];
	signed int i;

	for(i = 0; i<4; i++, num /= 10)
		v[i] = num % 10;
	return *(v + (num & 3))+'L';
}
```

- Se fosse invocata `my_fun(num)`, con il parametro `num` pari al tuo numero di matricola, cosa verrebbe ritornato?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_all(link head, int to_delete);
```

la quale elimini tutti i nodi con campo `value` uguale al parametro `to_delete` (tranne la testa della lista). La funzione deve restituire `TRUE` se e solo se almeno una cancellazione è avvenuta e `FALSE` altrimenti. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head`.

2018.06.28, corso B

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int half_is_greater(int * array, int size);
```

che restituisca `TRUE` se e solo se ogni elemento della metà sinistra dell'array è maggiore di ogni altro elemento della metà sinistra. In ogni altro caso la funzione dovrà resituire `FALSE`. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per `array` e `size`. Specificare quale gestione si intende realizzare per il caso `size` dispari.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdlib.h>

#define SIZE 4

char my_fun(unsigned int num) {
        signed char * v;
        signed int i;

        v = malloc(SIZE);
        for(i = 0; i<SIZE; i++, num /= 10)
                v[i] = num % 10;
        v++;
        return v[0];
}
```

- Se fosse invocata `my_fun(num)`, con il parametro `num` pari al tuo numero di matricola, cosa verrebbe ritornato?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_last(link head, int to_delete);
```

la quale elimini gli ultimi `to_delete` nodi a patto che la lista ne contenga almeno `to_delete`+1. La funzione deve restituire `TRUE` se e solo se la cancellazione è avvenuta e `FALSE` altrimenti. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head`.

2018.06.15, corso A

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int verify_sum(int * array, int size);
```

che restituisca `TRUE` se e solo se ogni elemento (tranne il primo) è uguale alla somma degli elementi precedenti; in ogni altro caso la funzione dovrà resituire `FALSE`. La soluzione ***DEVE*** fare uso di una funzione di appoggio `compute_sum(.....)` (con prototipo che potete definire a vostro piacimento) che calcoli la somma di una certa sequenza degli elementi di un array. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per `array` e `size`.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdlib.h>
#define DIV 100

int my_fun(unsigned int num, unsigned int len) {
	char * p;
	int i = 0;

	if ((num == 0) || len == 0)
		return -1;
	p = malloc(len);
	while (num > 0) {
		if (i >= len)
			return -1;
		p[i++] = num % DIV;
		num /= DIV;
	}
	i--;
	for (; i; i--)
		*(p+i-1) += *(p+i);
	return p[i];
}
```

- Se fosse invocata `my_fun(num,10)`, con il parametro `num` pari al tuo numero di matricola, quale valore verrebbe restituito al chiamante?
- Si faccia un esempio di `num` e `len` tali che il valore ritornato da `my_fun(num,len)` sia uguale zero

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
	int value;
	struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int insert_tail_if(link head, int to_add, int to_compare);
```

la quale aggiunga al fondo della lista un nodo con campo `value` uguale al parametro `to_add` ma solo se la lista contiene già l'elemento `to_compare`. La funzione deve restituire `TRUE` se e solo se l'inserimento è avvenuto e `FALSE` altrimenti. La soluzione ***DEVE*** fare uso di una funzione di appoggio `is_in_list(.....)` (con prototipo che potete definire a vostro piacimento) che restituisca `TRUE` se un elemento è presente in una lista. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head`.

2018.02.13, corso A

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int swap_array_elements(int * array, int size);
```

che scambi il primo con l'ultimo elemento dell'array, il secondo con il penultimo, eccetera. Si scriva anche una funzione di appoggio `swap(parametro1,parametro2)` che realizzi lo scambio dei due parametri. La funzione `swap_array_elements()` deve restituire TRUE se e solo se in almeno un caso gli elementi da scambiare sono uguali tra loro. Definite opportunamente `TRUE` e `FALSE` e gestite ogni possibile caso per `array` e `size`.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdlib.h>
int strange_fun(int m)
{
        char * p;
        int i;

        m = (m & 3) + 4;
        p = malloc(m*sizeof(*p));
        for(i=0; i<m; i++) {
                p[i] = m-i;
        }
        return *(p+i-1)+*p;
}
```

Se la funzione `strange_fun` fosse invocata passandogli il tuo numero di matricola come parametro, quale valore verrebbe restituito al chiamante?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int add_odd(link head, int to_add, int max_add);
```

la quale aggiunga dopo ogni elemento in posizione dispari (dopo il primo, dopo il terzo, eccetera) un nodo con campo `value` uguale al parametro `to_add`. La funzione deve restituire `TRUE` se e solo se il numero di inserimenti non sia stato maggiore del parametro `max_add`. Definite in modo opportuno `TRUE` e `FALSE` e gestite ogni possibile caso per la lista `head`.

2018.02.13, corso B

Esercizio 1 (4 punti)

Implementare la funzione con prototipo

```
int count_occurrences(int * array, int size, int element);
```

che conta il numero di occorrenze di `element` all'interno di `array` che ha dimensione `size`.

Implementare, inoltre, la funzione con prototipo

```
int analyze_array_elements(int * array, int size, int * max_occurrence);
```

che, utilizzando la funzione `count_occurrences` realizzata, restituisce `TRUE` se e solo se `array` contiene almeno un elemento il cui numero di occorrenze sia uguale ad 1 e `FALSE` altrimenti. Inoltre, assegna all'intero puntato dal parametro `max_occurrence` il valore dell'elemento che presenta il maggior numero di occorrenze.

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdio.h>
#define LEN 10
void strange_fun(int m)
{
        char v[LEN] = {'A'};
        int i;

        for (i=1; i<LEN; i++) {
                *(v+i) = *(v+i-1)+1;
        }
        m = (m+2) % 4;
        for (i=m; i< LEN; i+=4) {
                printf("%c", v[i]);
        }
}
```

Cosa verrebbe stampato dalla funzione funzione `strange_fun` se fosse invocata passandogli il tuo numero di matricola come parametro?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_even(link head);
```

la quale cancella e dealloca (cioè restituisce al gestore della memoria) tutti i nodi in posizioni pari (il secondo, il quarto, ...). La funzione deve restituire `TRUE` in caso almeno una cancellazione sia stata possibile e `FALSE` altrimenti. Definite in modo opportuno TRUE e FALSE e gestite ogni possibile caso per la lista `head`.

2018.01.22, corso A

Esercizio 1 (4 punti)

Si implementi la funzione con prototipo

```
int count_occurrences(int * array);
```

`count_occurrences()` è una funzione che restituisce `TRUE` se e solo se ogni elemento `N` del vettore `array` compare `N` volte e `FALSE` altrimenti. Definite opportunamente `TRUE` e `FALSE`, gestite il caso in cui `array` sia `NULL` ed il caso in cui `N` non sia maggiore di 0. Esempi:

- se il vettore `array` fosse: `2 3 2 3 3` allora la funzione restituirebbe `TRUE`;
- se il vettore `array` fosse: `2 3 2 1 3` allora la funzione restituirebbe `FALSE`;
- se il vettore `array` fosse: `3 2 3 2 1 3` allora la funzione restituirebbe `TRUE`;
- se il vettore `array` fosse: `-5 3 2 -7 3 2 1 3 -2` allora la funzione restituirebbe `TRUE`;

Esercizio 2 (2 punti)

Si scriva del codice che generi un errore di tipo "Segmentation fault" (o "Errore di segmentazione" in Italiano) e si indichi quale istruzione genera tale errore.

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
       int value;
       struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int delete_if(link head, int to_delete);
```

la quale cancella e dealloca (cioè restituisce al gestore della memoria) il primo nodo della lista il cui valore del campo `value` è uguale al parametro `to_delete` ma solo nel caso in cui tale nodo non sia la testa della lista. La funzione deve restituire `TRUE` in caso di avvenuta cancellazione e `FALSE` altrimenti. Definite in modo opportuno TRUE e FALSE e gestite il caso di lista vuota.

2018.01.22, corso B

Esercizio 1 (4 punti)

Si implementi la funzione con prototipo

```
int check_occurrences(char * str, int n_occurrences);
```

`check_occurrences()` è una funzione che restituisce `TRUE` se e solo se ogni carattere della stringa `str` compare un numero di volte uguale al parametro `n_occurences` e `FALSE` altrimenti. Definite opportunamente `TRUE` e `FALSE`, gestite il caso in cui `str` sia `NULL` ed il caso in cui `n_occurences` non sia maggiore di 0. Esempi:

- se la stringa `str` fosse: `dfdgdrdgd` e se `n_occurences` fosse 2 allora la funzione restituirebbe `FALSE`;
- se la stringa `str` fosse: `dfdffddf` e se `n_occurences` fosse 4 allora la funzione restituirebbe `TRUE`;
- se la stringa `str` fosse: `ddddffff` e se `n_occurences` fosse 2 allora la funzione restituirebbe `FALSE`;
- se la stringa `str` fosse: `abbacddc` e se `n_occurences` fosse 2 allora la funzione restituirebbe `TRUE`;

Esercizio 2 (2 punti)

Si consideri il seguente codice

```
#include <stdio.h>

#define LEN 4

int strange_fun(int m)
{
       int* p, v[LEN], i;

       m = m % 10;
       for(i=0; i<LEN; i++) {
              v[i] = (m+i) | 1;
       }
       p = v;
       for(i=0; i<LEN-1; i++) {
              *(p+1) += *p;
              p++;
       }
       return *p;
}

int main()
{
       int i;
```

```
        for (i=0; i< 10; i++) {
                printf("m␣%%␣10␣=␣%1d␣␣␣␣strange_fun(%1d)␣=␣%d\n",
                        i, i, strange_fun(i));
        }
}
```

Se la funzione `strange_fun` fosse invocata passandogli il tuo numero di matricola come parametro, quale valore verrebbe restituito al chiamante?

Esercizio 3 (4 punti)

Data la struttura

```
typedef struct node {
        int value;
        struct node * next;
} nodo;
typedef nodo* link;
```

implementare la funzione con prototipo

```
int add_if(link head, int to_add);
```

la quale aggiunge alla lista `head` un nodo con campo `value` uguale a 0 dopo un nodo che ha il campo `value` uguale a `to_add`. La funzione deve restituire `TRUE` in caso di avvenuta aggiunta e `FALSE` altrimenti. Definite in modo opportuno TRUE e FALSE e gestite il caso di lista vuota.